



Servidor MCP para compartir datos sísmicos

> Propuesta para el Prof. Rommel Contreras • Colaboración multi-investigador vía LLMs

Resumen para el Prof. Rommel

Usted puede instalar en su máquina un **servidor MCP** (Model Context Protocol) que exponga por red sus catálogo de eventos sísmicos y waveforms. Cada investigador, desde su propio ordenador y con su propia interfaz de IA (opencode, Claude Desktop u otra compatible con MCP), se conecta remotamente a su servidor y consulta los datos con lenguaje natural. El servidor **no requiere APIs de LLM**: solo sirve datos y herramientas; cada colega aporta su modelo.

Ventaja clave: ninguna máquina necesitará acceso físico a sus archivos. El dato vive en un único lugar (su PC) y los modelos de los demás investigadores lo consultan vía IA.

1 Cómo encaja MCP en la colaboración

MCP es un protocolo **cliente** ↔ **servidor** con dos transportes:

Transportes de MCP

Transporte	Úso	? Servidores remotos
stdio	Servidor en la misma máquina del cliente	No (local)
Streamable HTTP	Servidor accesible por red (URL con https://)	Sí

Para colaboración entre investigadores se usa Streamable HTTP. Punto importante: el servidor MCP **no necesita ninguna API de modelo**. Solo expone *datos + herramientas* (consulta de eventos, waveform, filtros por magnitud/profundidad). Cada investigador corre su propio cliente que *trae su propio LLM*. Es decir: una máquina en Venezuela aloja los datos; las LLMs de cada colega se conectan remotamente. El geo-bloqueo de Venezuela aplica a *APIs de LLM* (se resuelve con OpenRouter o Gemini), no a servir un endpoint HTTP local.

2 Topología propuesta

Red de colaboración

Investigador A • Investigador B • Investigador C

HTTPS + Bearer token → [FastMCP — Streamable HTTP] → /mcp puerto 8000 → datos sísmicos locales

↑

cloudflared (túnel) o nginx + Let's Encrypt

Cada investigador: **cliente MCP + su propio modelo**. El Prof. Rommel: **servidor + datos**.

En Venezuela la mayoría de ISPs usan **CGNAT** (sin IP pública ni port-forwarding), por lo que el acceso más sensato y gratuito es **Cloudflare Tunnel** (cloudflared): otorga una URL HTTPS sin IP estática. Alternativa clásica: nginx + **Let's Encrypt** si se dispone de IP pública o dominio.

3 Configuración del servidor (máquina del Prof. Rommel)

1. Instalar el framework:

```
1 pip install fastmcp
```

2. Código mínimo del servidor:

```
1 # servidor_sismico.py
2 from fastmcp import FastMCP
3
4 mcp = FastMCP("Sismologia-VE") # nombre visible para los clientes
5
6 @mcp.tool
7 def eventos(magnitud_min: float = 0.0,
8             prof_max_km: float = 300.0) -> list[dict]:
9     # Catálogo de eventos sísmicos filtrado por magnitud/profundidad
10     ... # lee el catálogo local (CSV/Parquet) y filtra
11
12 @mcp.tool
13 def waveform(id_evento: str) -> bytes:
14     # Devuelve el waveform del evento id_evento (SAC/mseed).
15     ... # sirve el archivo local
16
17 if __name__ == "__main__":
18     mcp.run(transport="streamable-http",
```

```
19         host="0.0.0.0", port=8000)
```

3. Exponer a internet:

```
1  cloudflared tunnel --url http://127.0.0.1:8000
2  # devuelve p. ej. https://xxxx.trycloudflare.com
3  # -> el endpoint MCP es: https://xxxx.trycloudflare.com/mcp
```

4 Configuración de los clientes (cada investigador)

opencode (en `opencode.json` global o del proyecto):

```
1  {
2    "mcp": {
3      "sismica": {
4        "type": "remote",
5        "url": "https://xxxx.trycloudflare.com/mcp",
6        "headers": { "Authorization": "Bearer <token-compartido>" },
7        "enabled": true
8      }
9    }
10 }
```

Claude Desktop (`claude_desktop_config.json`):

```
1  {
2    "mcpServers": {
3      "sismica": {
4        "type": "http",
5        "url": "https://xxxx.trycloudflare.com/mcp",
6        "headers": { "Authorization": "Bearer <token-compartido>" }
7      }
8    }
9  }
```

El token se reparte entre los investigadores por canal cifrado; para un grupo cerrado es suficiente y puede rotarse periódicamente.

5 Consideraciones

 Seguridad

Exigir **HTTPS** (el túnel lo da gratis) y **Bearer token**; no exponer el puerto 8000 crudo a internet. Para un grupo como una comunidad de investigadores es una solución práctica.

 Concurrencia y carga

Varios clientes golpean el servidor a la vez. Si los waveforms son pesados, cachear en disco o servir una versión decimada por defecto; a menor volumen de datos transferidos, menor consumo de tokens de los modelos y menor uso de su RAM (recordar que la PC del receptor tiene 4 GB).

 Disponibilidad y respaldo

Si la máquina del Prof. Rommel se apaga, los datos dejan de estar disponibles. Opción robusta: publicar el catálogo en GitHub o Hugging Face Dataset, de modo que cada investigador apunte su MCP a su propio clon y el servidor central solo orqueste los metadatos.

 Verificación del paquete

Igual que ocurrió con @platformio/mcp (que no existía; el real era platformio-mcp), conviene confirmar el nombre y versión del paquete MCP en el índice antes de asumir: `pip index versions mcp` y revisar PyPI.

Próximo paso acordado: estudiar la construcción de un **prototipo funcional en el repositorio ELEC-TRONICA** siguiendo la arquitectura de 3 capas — `directiva directives/servidor_sismico.yaml` + orquestador + `execution/servidor_sismico.py` con un catálogo sintético de ejemplo — para que el Prof. Rommel lo clone y solo cambie la ruta a sus datos reales.